

A Guide to Using Raw Sockets

In this tutorial, let's take a look at how raw sockets can be used to receive data packets and send those packets to specific user applications, bypassing the normal TCP/IP protocols.



If you have no knowledge of the Linux kernel, yet are interested in the contents of network packets, raw sockets are the answer. A raw socket is used to receive raw packets. This means packets received at the Ethernet layer will directly pass to the raw socket. Stating it precisely, a raw socket bypasses the normal TCP/IP processing and sends the packets to the specific user application (see Figure 1).

A raw socket vs other sockets

Other sockets like stream sockets and data gram sockets receive data from the transport layer that contains no headers but only the payload. This means that there is no information about the source IP address and MAC address. If applications running on the same machine or on different machines are communicating, then they are only exchanging data.

The purpose of a raw socket is absolutely different. A raw socket allows an application to directly access lower level protocols, which means a raw socket receives un-extracted packets (see Figure 2). There is no need to provide the port and IP address to a raw socket, unlike in the case of stream and datagram sockets.

Network packets and packet sniffers

When an application sends data into the network, it is

processed by various network layers. Before sending data, it is wrapped in various headers of the network layer. The wrapped form of data, which contains all the information like the source and destination address, is called a network packet (see Figure 3). According to Ethernet protocols, there are various types of network packets like Internet Protocol packets, Xerox PUP packets, Ethernet Loopback packets, etc. In Linux, we can see all protocols in the *if_ether.h* header file (see Figure 4).

When we connect to the Internet, we receive network packets, and our machine extracts all network layer headers and sends data to a particular application. For example, when we type *www.google.com* in our browser, we receive packets sent from Google, and our machine extracts all the headers of the network layer and gives the data to our browser.

By default, a machine receives those packets that have the same destination address as that of the machine, and this mode is called the non-promiscuous mode. But if we want to receive all the packets, we have to switch into the promiscuous mode. We can go into the promiscuous mode with the help of *ioctl*s.

If we are interested in the contents or the structure of the headers of different network layers, we can access these with the help of a packet sniffer. There are various packet sniffers